

## Week\_4

**Task 1:** Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

| ALGORITHM: Parent Creates Multiple Children and Synchronizes using waitpid() |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.                                                                           | START                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 2.                                                                           | Declare an array <code>child[NUM_CHILDREN]</code> to store child PIDs and an integer <code>status</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 3.                                                                           | Print a message indicating the parent process has started along with its PID.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 4.                                                                           | <div><div>For <math>i \leftarrow 0</math> to <math>\text{NUM\_CHILDREN} - 1</math> do</div><div><div>4.1 Call <code>fork()</code> and store the return value in <code>child[i]</code>.</div><div>4.2 If <code>fork() &lt; 0</code> then</div><div>4.3     Print error message and terminate the program.</div><div>4.4 Else if <code>fork() == 0</code> then     // Child process</div><div><div>4.4.1 Print message: "Child <math>i + 1</math> started" along with child PID.</div><div>4.4.2 Sleep for <math>(i + 1) \times 2</math> seconds (to create different completion times).</div><div>4.4.3 Print message: "Child <math>i + 1</math> completed".</div><div>4.4.4 Exit the child process with exit status <math>(i + 1)</math>.</div></div></div></div> |
| 5.                                                                           | End For                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 6.                                                                           | Print message: Parent waiting for children using <code>waitpid()</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 7.                                                                           | For $i \leftarrow 0$ to $\text{NUM\_CHILDREN} - 1$ do                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 8.                                                                           | Call <code>waitpid(child[i], &amp;status, 0)</code> and store the return value in <code>pid</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 9.                                                                           | If <code>waitpid() == -1</code> then                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 10.                                                                          | Print error message and terminate the program.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 11.                                                                          | Else                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 12.                                                                          | If <code>WIFEXITED(status)</code> is true then                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 13.                                                                          | Print: Child with PID <code>pid</code> terminated with exit status <code>WEXITSTATUS(status)</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 14.                                                                          | End If                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 15.                                                                          | End If                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 16.                                                                          | End For                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 17.                                                                          | Print message: All child processes completed. Parent exiting.<br>STOP                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

```
GNU nano 7.2 practical4_task1.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

#define NUM_CHILDREN 3

int main()
{
    pid_t child[NUM_CHILDREN];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < NUM_CHILDREN; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            printf("Fork failed.\n");
            return 1;
        }

        if (child[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d completed. PID = %d\n", i + 1, getpid());
            exit(i + 10);
        }
    }

    printf("\nAll %d child processes have been created.\n", NUM_CHILDREN);
    printf("Using wait():\n");

    pid_t finished = wait(&status);
    if (WIFEXITED(status))
    {
        printf("wait() detected child PID %d finished with exit status %d\n",
            finished, WEXITSTATUS(status));
    }

    printf("Using waitpid():\n");
    for (int i = 0; i < NUM_CHILDREN; i++)
    {
        if (child[i] == finished)
            continue;

        waitpid(child[i], &status, 0);

        if (WIFEXITED(status))
        {
            printf("waitpid() detected child PID %d finished with exit status %d\n",
                child[i], WEXITSTATUS(status));
        }
    }

    printf("\nParent process completed.\n");
    return 0;
}
```

```
sindhuja@SINDHUJA: ~/pract x + v
sindhuja@SINDHUJA:~$ mkdir practical_week4
sindhuja@SINDHUJA:~$ cd practical_week4
sindhuja@SINDHUJA:~/practical_week4$ pwd
/home/sindhuja/practical_week4
sindhuja@SINDHUJA:~/practical_week4$ nano practical4_task1.c
sindhuja@SINDHUJA:~/practical_week4$ ls
practical4_task1.c
sindhuja@SINDHUJA:~/practical_week4$ gcc practical4_task1.c -o practical4_task1
sindhuja@SINDHUJA:~/practical_week4$ ls
practical4_task1 practical4_task1.c
sindhuja@SINDHUJA:~/practical_week4$ ./practical4_task1
Parent Process Started
Parent PID: 545

Child 1 started. PID = 546, Parent PID = 545

All 3 child processes have been created.

Child 2 started. PID = 547, Parent PID = 545
Using wait():
Child 3 started. PID = 548, Parent PID = 545
Child 1 completed. PID = 546
wait() detected child PID 546 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 547
waitpid() detected child PID 547 finished with exit status 11
Child 3 completed. PID = 548
waitpid() detected child PID 548 finished with exit status 12

Parent process completed.
sindhuja@SINDHUJA:~/practical_week4$ |
```

Sample Output:

Parent Process Started

Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521

Child 2 started. PID = 4523, Parent PID = 4521

Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using wait():

Child 1 completed. PID = 4522

wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523

Child 3 completed. PID = 4524

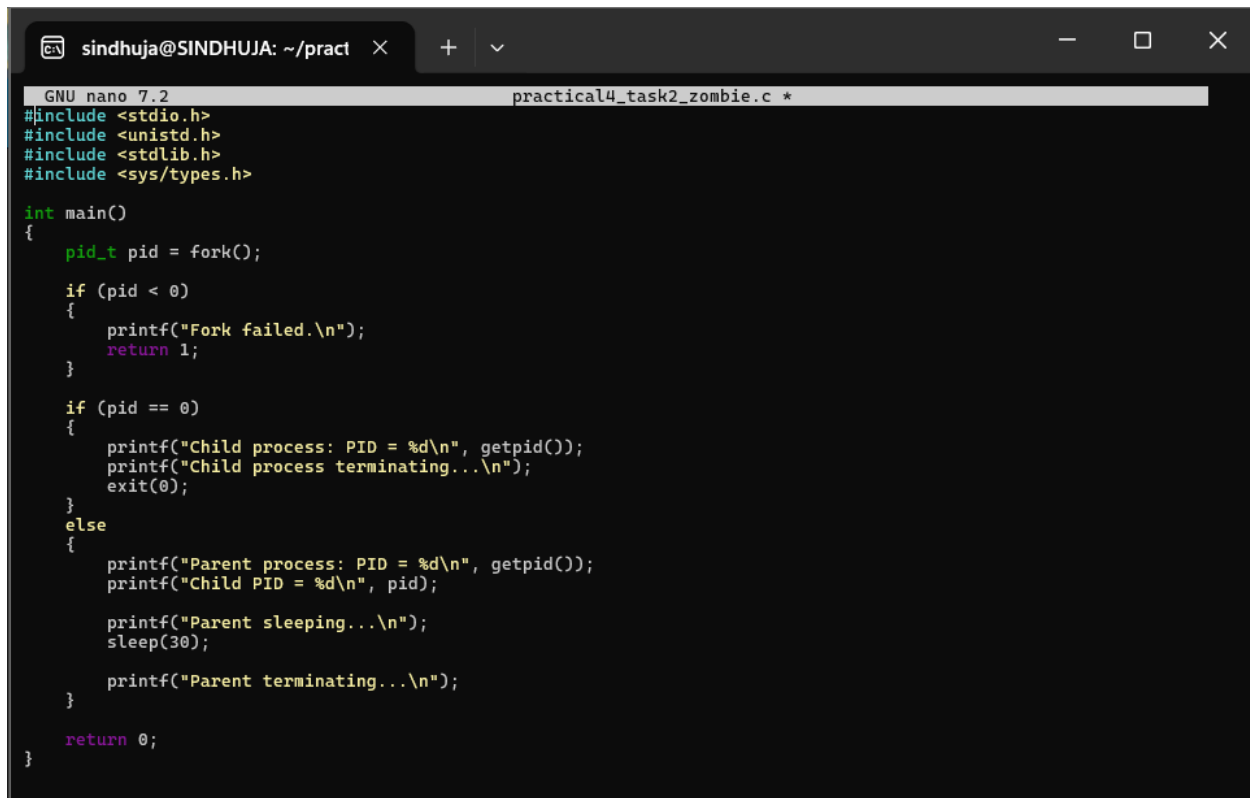
waitpid() detected child PID 4522 finished with exit status 10

waitpid() detected child PID 4523 finished with exit status 11

waitpid() detected child PID 4524 finished with exit status 12

Parent process completed.

**Task 2:** Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.



```
GNU nano 7.2 practical4_task2_zombie.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping...\n");
        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

```
sindhujaa@SINDHUJA:~/practical_week4$ nano practical4_task2_zombie.c
sindhujaa@SINDHUJA:~/practical_week4$ gcc practical4_task2_zombie.c -o practical4_task2_zombie
sindhujaa@SINDHUJA:~/practical_week4$ ./practical4_task2_zombie
Parent process: PID = 581
Child PID = 582
Parent sleeping...
Child process: PID = 582
Child process terminating...
```

```
sindhujaa@SINDHUJA: ~/pract  ×  +  ▾
GNU nano 7.2 practical4_task2_waitpid.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, &status, 0);

        printf("waitpid() collected child exit status.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}
```

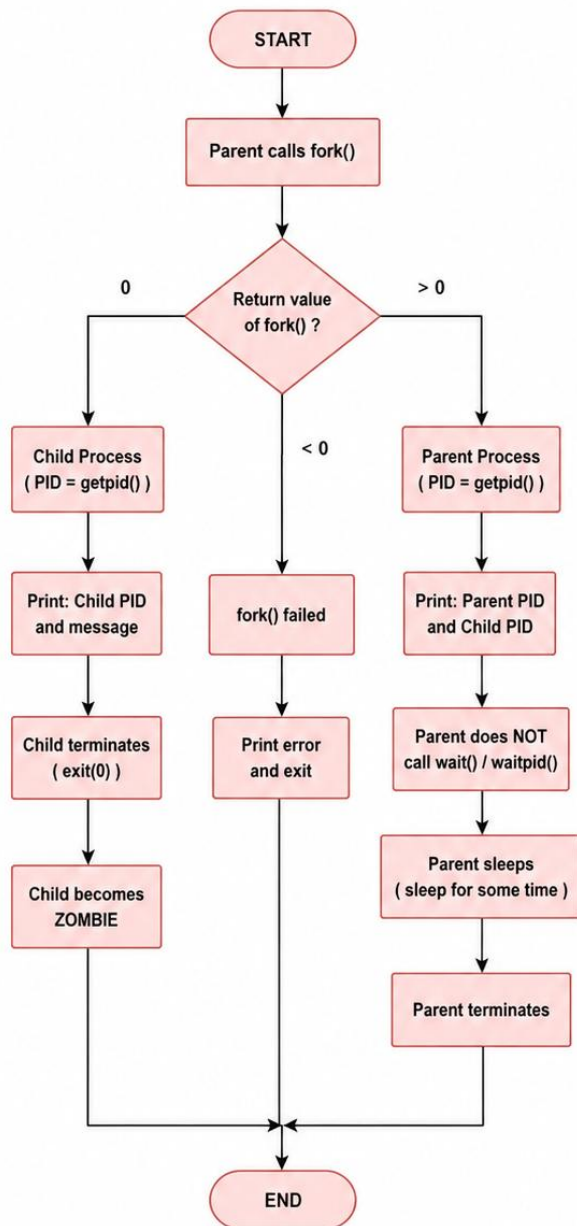
^G Help    ^O Write Out    ^W Where Is    ^K Cut    ^T Execute    ^C Location    M-U Undo  
 ^X Exit    ^R Read File    ^\_ Replace    ^U Paste    ^J Justify    ^\_ Go To Line    M-E Redo

```
practical4_task1 practical4_task1.c
sindhujaa@SINDHUJA:~/practical_week4$ nano practical4_task2_zombie.c
sindhujaa@SINDHUJA:~/practical_week4$ gcc practical4_task2_zombie.c -o practical4_task2_zombie
sindhujaa@SINDHUJA:~/practical_week4$ ./practical4_task2_zombie &
[1] 938
sindhujaa@SINDHUJA:~/practical_week4$ Parent process: PID = 938
Child PID = 939
Parent sleeping for 60 seconds...
Child process: PID = 939
Child process terminating...
sindhujaa@SINDHUJA:~/practical_week4$ ps -el | grep practical4_task2_zombie
sindhujaa@SINDHUJA:~/practical_week4$ ps -el | grep 938
 0 S 1000 938 822 0 80 0 - 672 hrtime pts/0 00:00:00 practical4_task
 1 Z 1000 939 938 0 80 0 - 0 - pts/0 00:00:00 practical4_task
sindhujaa@SINDHUJA:~/practical_week4$ Parent terminating...

[1]+ Done ./practical4_task2_zombie
sindhujaa@SINDHUJA:~/practical_week4$ nano practical4_task2_waitpid.c
sindhujaa@SINDHUJA:~/practical_week4$ gcc practical4_task2_waitpid.c -o practical4_task2_waitpid
sindhujaa@SINDHUJA:~/practical_week4$ ./practical4_task2_waitpid
Parent process: PID = 996
Child PID = 997
Child process: PID = 997
Child process terminating...
waitpid() collected child exit status.
Parent terminating...
sindhujaa@SINDHUJA:~/practical_week4$ ps -el | grep practical4_task2_waitpid
sindhujaa@SINDHUJA:~/practical_week4$
```

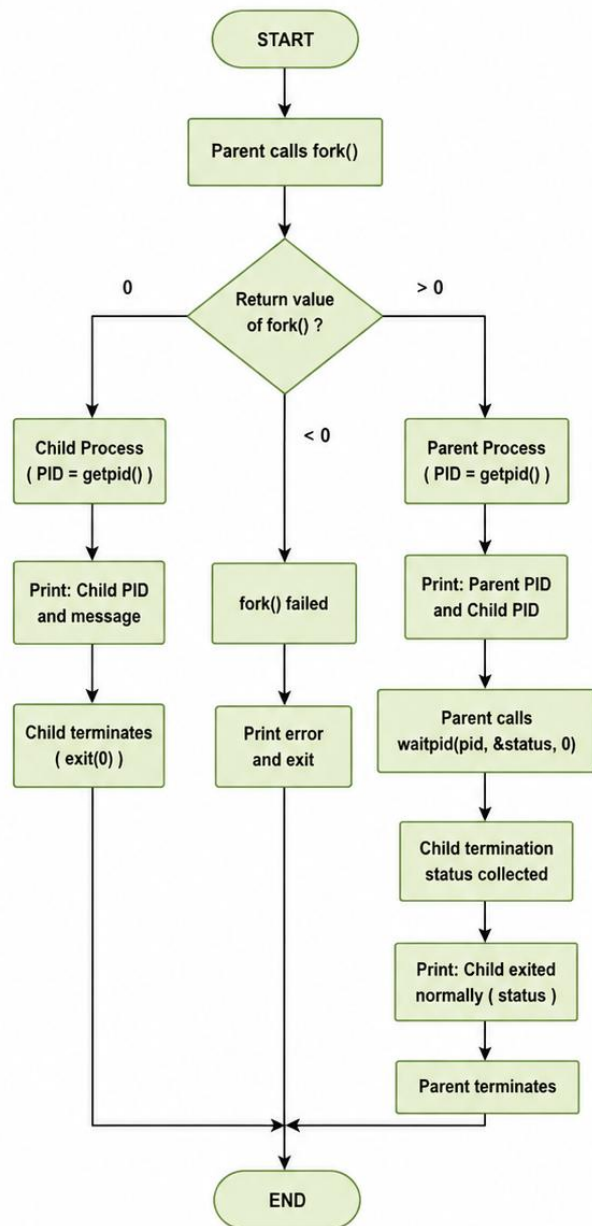
Flow Chart:

**FLOWCHART 1: CREATE ZOMBIE PROCESS**  
(Parent does NOT call waitpid())



During the time parent is sleeping,  
child is shown in process table as Z (zombie / <defunct>)

**FLOWCHART 2: ELIMINATE ZOMBIE PROCESS**  
(Parent calls waitpid())



Since parent calls waitpid(),  
child does NOT remain a zombie.

Sample Output:

Parent process: PID = 158

Child process: PID = 159  
Child PID = 159  
Child process terminating...  
Parent sleeping...  
Parent terminating...

Ps -el

| F | S        | UID  | PID | PPID | C | PRI | NI | ADDR     | SZ | WCHAN | TTY | TIME     | CMD      |
|---|----------|------|-----|------|---|-----|----|----------|----|-------|-----|----------|----------|
| 0 | S        | 0    | 1   | 0    | 0 | 80  | 0  | - 2238 ? | ?  |       |     | 00:00:00 | init     |
| 0 | S        | 0    | 8   | 1    | 0 | 80  | 0  | - 2238 ? |    | tty1  |     | 00:00:00 | init     |
| 0 | S        | 1000 | 9   | 8    | 0 | 80  | 0  | - 3590 ? |    | tty1  |     | 00:00:00 | bash     |
| 0 | S        | 0    | 141 | 1    | 0 | 80  | 0  | - 2238 ? |    | tty2  |     | 00:00:00 | init     |
| 0 | S        | 1000 | 142 | 141  | 0 | 80  | 0  | - 3527 ? |    | tty2  |     | 00:00:00 | bash     |
| 0 | S        | 1000 | 158 | 9    | 0 | 80  | 0  | - 2680 ? |    | tty1  |     | 00:00:00 | week4_T2 |
| 0 | <b>Z</b> | 1000 | 159 | 158  | 0 | 80  | 0  | - 0 ?    |    | tty1  |     | 00:00:00 | week4_T2 |
| 0 | R        | 1000 | 160 | 142  | 0 | 80  | 0  | - 4086 ? |    | tty2  |     | 00:00:00 | ps       |